Together for Tomorrow!
Enabling People
Education for Future Generations

AI Course

Capstone Project Final Report

For students (instructor review required)

©2025 SAMSUNG. All rights reserved.
Samsung Electronics Corporate Citizenship Office holds the copyright of this document.
This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this document other than the curriculum of Samsung Innovation Campus, you must receive written consent from copyright holder.

VietTravel AI: A Grounded Travel Assistant and Itinerary Planner for Vietnam

21/10/2026

AI Course
Samsung Innovation Campus

Class:
SIC-AI-N02

Instructor:
Dr. Trần Quang Quý

Travel AI Team

Members:

Nguyễn Minh Phương
Thân Phú Cường

Abstract

VietTravel AI is a Flask-based travel assistant for discovering places in Vietnam and preparing one-day itinerary drafts. The repository combines a natural-language travel-search path with a separate itinerary-planning workflow. The search path detects a supported language, extracts travel-related features, queries a PostgreSQL staging table populated from OpenStreetMap (OSM), and asks a configured language-model provider to compose a response when credentials are available. The itinerary endpoint retrieves candidate points of interest (POIs), applies a deterministic schedule planner by default, checks the resulting draft against explicit identity, time, and coordinate rules, and returns the draft without saving it. A separately authenticated confirmation endpoint revalidates and persists a draft against the relational application schema.

This report documents what is present in the source tree and the accompanying data manifests. The available nationwide OSM processing summary reports 30,941 filtered candidates and 30,839 unique identifiers, although its duplicate-count fields do not reconcile exactly. The repository includes a 15-record pilot fixture and a nine-case unittest module using mocked provider and database calls. No reproducible model benchmark, load-test result, or executed test log is included with the project snapshot. Accordingly, the experimental chapter separates recorded dataset evidence and declared test cases from measurements that remain to be performed. The report describes the implemented system, its operating assumptions, known limitations, and a reproducible evaluation plan without presenting unverified scores as results.

Keywords: travel assistant, itinerary planning, OpenStreetMap, PostgreSQL, Flask, deterministic validation, Vietnam tourism.

Contents

Abstract i

List of Abbreviations vii

1. Introduction 1

1.1. Background . . . 1
1.2. Problem Statement . . . 1
1.3. Objectives . . . 2
1.4. Scope and Non-Goals . . . 2
1.5. Method and Evidence Policy . . . 3
1.6. Contributions and Report Organization . . . 3

2. Related Work 4

2.1. Tourism Recommender Systems . . . 4
2.1.1. Content-based and collaborative filtering . . . 4
2.1.2. Context-aware and constraint-based planning . . . 4
2.2. Large Language Models and Retrieval Grounding . . . 5
2.3. Multi-Agent Orchestration . . . 5
2.4. Geospatial Search and Route Planning . . . 6
2.5. Open Geographic Data and Data Quality . . . 6
2.6. Comparison and Research Gap . . . 6

3. Dataset and Pre-processing 8

3.1. Data Sources and Scope . . . 8
3.2. Candidate Filtering and Standardization . . . 8
3.3. Recorded Dataset Profile . . . 9
3.3.1. A reconciliation issue in the manifest . . . 9
3.4. Relational Representation . . . 10
3.5. Spatial and Text Pre-processing . . . 11
3.6. Data Quality, Licensing, and Reproducibility . . . 11
3.7. Summary . . . 12

4. Model Configuration 13

4.1. Application Architecture . . . 13
4.2. Natural-Language Travel Search . . . 14
4.3. Itinerary Draft Workflow . . . 14
4.4. Validation and Route Estimates . . . 15
4.5. Model and Provider Configuration . . . 16
4.6. Persistence, Accounts, and Supporting Services . . . 16
4.7. Security and Operational Considerations . . . 17
4.8. Development Configuration . . . 17

5. Experimental Results and Discussion 19

5.1. Evaluation Scope and Evidence Available . . . 19
5.2. Dataset Processing Results . . . 19
5.3. Functional Test Design in the Repository . . . 19
5.4. Implemented Behavior Observed by Source Inspection . . . 20
5.5. Proposed Retrieval Evaluation . . . 21
5.6. Proposed Itinerary and Answer Evaluation . . . 21
5.7. Proposed Runtime and Reliability Evaluation . . . 22
5.8. Discussion of Current Results . . . 22
5.9. Recommended Evaluation Sequence . . . 23
5.10. Chapter Summary . . . 23

References 24

A. Data Dictionary and Reproducibility Notes 26

A.1. Staging Record Fields . . . 26
A.2. Import Reproduction Checklist . . . 27

B. API Surface and Request Boundaries 28

B.1. Core Travel and Itinerary Routes . . . 28
B.2. Draft Request Validation . . . 28
B.3. Draft Confirmation Boundary . . . 29

C. Regression Test and Evaluation Register 30

C.1. Existing Regression Cases 30

C.2. Additional Tests Recommended Before Release 31

C.3. Evaluation Record Template 31

C.4. Local Development Commands 31

C.5. Closing Note 32

6. Instructor Review and Comment 33

List of Figures

4.1 Implemented application paths and external dependencies 13

List of Tables

2.1 Capability comparison for travel-assistant design 7

3.1 Layer-level filtering values recorded in the ingestion summary 9

3.2 Unique POI counts by standardized category group 9

3.3 Principal relational entities in the inspected application models 10

4.1 Provider and database settings visible in the source configuration 16

5.1 Regression test cases declared in the repository 20

A.1 OSM staging field dictionary 26

B.1 Core travel-search and itinerary endpoints 28

C.1 Test-to-requirement mapping for the current test module 30

C.2 Evaluation record template for a future run 32

6.1 Capstone Project Evaluation Rubric and Instructor Feedback 33

List of Abbreviations

Abbreviation	Definition
AI	Artificial intelligence
API	Application programming interface
CORS	Cross-origin resource sharing
GPS	Global Positioning System
Haversine	Great-circle distance formula for spherical coordinates
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large language model
MAS	Multi-agent system
OSM	OpenStreetMap
OTP	One-time password
POI	Point of interest
REST	Representational State Transfer
SQL	Structured Query Language
SMTP	Simple Mail Transfer Protocol
UI	User interface
VND	Vietnamese dong

1. Introduction

1.1. Background

Travel planning brings together information that is usually scattered across maps, business listings, booking sites, public transport pages, and local recommendations. A traveler may begin with a broad question such as where to eat near a landmark, then refine it with a location, a date, a preferred pace, or a limited amount of time. A useful assistant therefore needs to do more than produce fluent text. It must retrieve places that exist, preserve their geographic identity, make the limits of incomplete information visible, and present a schedule that can be understood and changed.

This problem is especially concrete in a destination catalogue assembled from community-maintained geographic data. OpenStreetMap is a collaborative source of geographic features, and its coverage and tagging vary by place and feature type [1]. A record can have a name and coordinates while lacking opening hours, prices, or a street address. A system that fills those gaps with plausible-sounding guesses may appear helpful while making a traveler less informed. The design challenge is thus twofold: use structured records to ground place identity, and retain explicit uncertainty where records are incomplete.

Large language models (LLMs) have made it practical to interpret open-ended questions and produce natural-language responses. Transformer models provide a flexible foundation for this interaction [2], [3]. However, a model's linguistic fluency does not guarantee that an attraction exists, that a price is current, or that a proposed trip can be completed on time. Retrieval-augmented generation (RAG) addresses part of this problem by conditioning generation on retrieved evidence [4], [5]. It does not, by itself, make the data current or complete, prove that the retrieval is relevant, or validate a schedule.

The repository examined in this report, *ChatbotTravel_server* (also branded in the source documentation as *VietTravel AI*), is a Python web application for travel search and itinerary management. It uses Flask and Flask-RESTX to expose HTTP endpoints, SQLAlchemy to model relational data, and PostgreSQL as its intended database. A local web client is served from the same Flask application. The data pipeline prepares tourism-related points of interest (POIs) from Vietnam OSM shapefiles and loads normalized records into a PostgreSQL staging table.

There are two related but distinct interaction paths in the codebase. The travel-chat path detects language, extracts intent and features, searches the staging data, and uses configured text-generation providers when available. The itinerary path accepts a structured planning request, retrieves candidate POIs, builds a draft with a deterministic scheduling routine by default, validates it, and returns it without persistence. A separate confirmation operation checks a bearer token and persists a confirmed schedule. Keeping these paths distinct is important: the source code does not implement one autonomous graph in which every chatbot request passes through the itinerary planner.

1.2. Problem Statement

The project addresses a practical engineering question:

> How can a web-based travel assistant combine conversational access to destination data with

a constrained itinerary workflow while making the source and certainty of place, schedule, and route information explicit?

The question decomposes into several implementation concerns:

1. **Geographic grounding.** A generated answer should use POIs retrieved from the requested geographic scope and should retain the source identifier and coordinates needed to trace a recommendation back to the catalogue.
2. **Incomplete attributes.** Community records do not consistently contain price, rating, contact information, or opening hours. Unknown values must remain unknown rather than being silently substituted with invented values.
3. **Schedule validity.** A generated one-day schedule needs unique stops, valid coordinates, properly formatted time intervals, no overlaps, and a reasonable upper bound on the day span. Geographic distance and actual driving time are different quantities and should not be conflated.
4. **Persistence and identity.** A plan assembled from OSM identifiers is not automatically a record in the application's curated `Attractions` table. The draft and confirmation stages must preserve this distinction and verify the authenticated user before writing.
5. **Evaluability.** A useful capstone report should separate source-code behavior, dataset manifests, and test definitions from measured model quality or runtime performance. These evidence types answer different questions.

1.3. Objectives

The engineering objectives reflected in the inspected source tree are:

1. Provide a REST API and browser interface for travel questions and itinerary planning.
2. Prepare Vietnam OSM data as named POIs with standardized categories, coordinates, and source metadata.
3. Search PostgreSQL by text or category, and by radius when coordinates are available.
4. Build one-day itinerary drafts from retrieved candidates, using the deterministic planner by default.
5. Validate each draft in application code and report errors separately from warnings.
6. Let verified users confirm a draft and save it transactionally.
7. State implementation limits and propose an evaluation plan without presenting unrun experiments as results.

1.4. Scope and Non-Goals

The report covers the contents of the repository snapshot, including the Flask entry point, application factory, controllers, service modules, ORM models, SQL migrations, data-import scripts, OSM files, the pilot JSON fixture, and the current regression-test source. The report describes the intended local development setup from `docker-compose.dev.yml` and `requirements.txt`.

The work examined here does not establish that a particular PostgreSQL instance currently contains all records described in the data manifest. It does not provide an independently labeled relevance dataset, a human evaluation panel, a reproducible load-test log, or a model-quality benchmark. The source includes optional integration code for Google Gemini in the planner and a separate provider path in the chatbot; the itinerary controller explicitly calls the planner with external API execution disabled. Consequently, the default itinerary result is produced by a deterministic heuristic rather than by a live Gemini call. The codebase also contains optional ChromaDB-related code, but the itinerary orchestrator explicitly disables it when it searches PostgreSQL.

The project should therefore be interpreted as a working prototype with implementable interfaces and validation rules, not as evidence of production-scale coverage or measured recommendation quality. In particular, a static validator can reject malformed data but cannot guarantee a correct road route, current business hours, accessibility, price, safety, or user satisfaction.

1.5. Method and Evidence Policy

This report was prepared by inspecting the repository's source files and included documentation. Its implementation claims refer to named modules and executable paths. Its dataset counts refer to `data/osm_full_ingestion_summary.json` and are treated as manifest values rather than a live database count. Its testing discussion refers to the test cases declared in `tests/test_multi_agent_flow.py`; tests were not executed while preparing this report, and no pass/fail result is inferred from their presence.

Evidence is classified in three ways throughout the report:

Recorded A value appears in a committed data file, configuration file, migration, or test definition.

Implemented A behavior is traceable to a function or controller path in the source code.

Unmeasured A performance, quality, or operational outcome for which the snapshot provides no reproducible run output.

This policy is particularly relevant to Chapter 5: the manifest provides descriptive counts, and test source provides test intentions, but neither is equivalent to an executed end-to-end benchmark.

1.6. Contributions and Report Organization

The repository's principal engineering contribution is an end-to-end structure connecting geographic data preparation to a web-facing travel application. The data loader filters and normalizes selected OSM layers; a relational staging model preserves the source identifier and source attributes; search services provide database-backed candidates; the planner produces a time-ordered draft; and a validator checks a defined set of structural and scheduling constraints. Authentication and itinerary persistence are implemented as complementary application features.

Chapter 2 reviews recommendation, grounding, agent orchestration, and geographic planning concepts relevant to the implemented design. Chapter 3 describes the available OSM evidence, import rules, schema, and data-quality caveats. Chapter 4 explains the running architecture and its model/provider configuration. Chapter 5 reports only reproducible evidence found in the repository, discusses limitations, and proposes a future evaluation protocol. Appendices provide a compact data dictionary, endpoint inventory, and the test and evaluation checklist.

2. Related Work

2.1. Tourism Recommender Systems

Tourism recommendation is a context-rich decision-support problem. A traveler chooses combinations of attractions, food, accommodation, and transport under limits on place, date, time, cost, party needs, and personal interest. These decisions differ from a simple item-ranking task because the selected items form a trip: order matters, travel consumes time, and several pieces of information may be uncertain. Surveys of tourism recommenders describe content-based, collaborative, knowledge-based, and context-aware approaches as complementary families rather than a single universal solution [6], [7].

2.1.1. Content-based and collaborative filtering

Content-based filtering ranks items using item attributes that resemble a user's stated or learned preferences. If POIs are represented by feature vectors $\mathbf{x}_i$, and a profile by $\mathbf{u}$, a common similarity is

$$\cos(\mathbf{u}, \mathbf{x}_i) = \frac{\mathbf{u}^\mathsf{T}\mathbf{x}_i}{\|\mathbf{u}\|_2 \|\mathbf{x}_i\|_2}. \tag{2.1}$$

This family can recommend a new item if its metadata is available, but its quality depends on descriptive and consistent item features. A sparse OSM record with only a name, class, and coordinate can support category and distance filtering, but it provides little evidence about atmosphere, accessibility, or suitability for a particular traveler. The project therefore has a searchable catalogue, but the inspected implementation does not include a learned user-preference profile or a trained recommendation model.

Collaborative filtering estimates preference from interactions among users and items. In tourism settings, interactions can be sparse because many users visit a destination infrequently, and a new user or new destination may have little history. Personalized-tourism research explores strategies that combine interests, popularity, and context to address some of these gaps [8]. The repository's available ORM model records accounts and itineraries, but the source inspected for this report does not implement a collaborative ranking algorithm or a rating-feedback loop. It would be inaccurate to describe the current system as a trained recommender in the matrix-factorization sense.

2.1.2. Context-aware and constraint-based planning

Context-aware systems use factors such as time, location, weather, group composition, or current activity in addition to item preference [7]. A travel itinerary also has hard constraints that are naturally expressed as feasibility checks: stops should not overlap, visits should fit within available hours, and locations should belong to the supported region. These constraints can be enforced independently of a language model.

An itinerary formulation can be written as the selection and ordering of a subset $\mathcal{S}$ of candidate POIs. A generalized objective may reward user relevance while penalizing time and distance:

$$\max_{\mathcal{S},\pi} \sum_{i \in \mathcal{S}} u_i - \lambda_d \sum_{j=1}^{|\mathcal{S}|-1} d_{\pi_j,\pi_{j+1}} - \lambda_t \sum_{j=1}^{|\mathcal{S}|} t_{\pi_j}, \tag{2.2}$$

subject to budget, time-window, and trip-duration constraints. Here u_i is an item utility, π is the visit order, d is a distance, and t is a visit duration. This equation describes a useful design space; it is not the optimization objective implemented by the current planner. The current deterministic planner uses a fixed set of day-time slots and a simple dining-versus-sightseeing preference when selecting from the supplied candidate list.

2.2. Large Language Models and Retrieval Grounding

The Transformer architecture uses attention to model dependencies across sequences and underlies many contemporary language models [2]. Large models can interpret varied wording and generate coherent explanations, which is attractive for conversational interfaces. Few-shot language models further illustrate how prompting can guide task behavior without fitting task-specific model parameters [3]. Yet a generated answer is not automatically grounded in a current local dataset. A model may confidently state a fact unsupported by the records supplied to it. Retrieval-augmented generation combines a learned generator with external evidence retrieved at response time [4]. Dense retrieval embeds queries and documents into a shared vector space; Sentence-BERT is one approach for producing sentence embeddings suited to semantic comparison [9]. Sparse retrieval such as BM25 provides a complementary lexical signal for exact terms [10]. A hybrid retriever can combine semantic similarity, exact text, structured category filters, and geography, but each component requires an implementation and an evaluation set. A dependency on a vector database alone does not establish that such a hybrid score is being used.

The repository contains dependencies for ChromaDB and Sentence-Transformers, and `poi_search_tool.py` includes an optional, read-only Chroma lookup. However, the itinerary orchestrator explicitly calls PostgreSQL search with `include_chromadb=False`. Its primary retrieval path uses SQL text/category filters and, when a center coordinate and radius are supplied, geospatial filtering. The report therefore treats the itinerary flow as structured SQL retrieval, not as a deployed dense-vector RAG pipeline. In the chat path, PostgreSQL results are passed into a language-model response function; optional Google Search grounding is a separate provider capability and depends on configuration and provider availability. Search grounding does not make OSM metadata complete and cannot substitute for a domain-specific evaluation.

2.3. Multi-Agent Orchestration

Multi-agent approaches assign distinct roles to interacting language-model or software agents. Frameworks such as AutoGen and MetaGPT study communication patterns and role specialization for complex tasks [11], [12]. ReAct interleaves reasoning and tool actions [13], while Reflexion introduces feedback from evaluating earlier attempts [14]. These ideas motivate decomposing a travel workflow into retrieval, planning, routing, and validation responsibilities.

The itinerary workflow runs a sequence of application modules. `orchestrator.py` validates the request, searches staging POIs, sends candidates to the planner, checks the draft, and retries when blocking errors remain. The API controller sets `execute_api=False`, so the endpoint uses the deterministic Python planner by default. A direct caller can enable the optional Gemini branch in `planner_agent.py` when the SDK and credentials are available; the API does not enable it.

These roles are software modules with explicit inputs and outputs, rather than concurrent autonomous agents. This structure makes each step inspectable: missing search results can be reported, unsupported

OSM identifiers can be rejected, and warnings can be returned separately. The planner does not learn from user feedback, and the validator can enforce only rules that have been encoded.

2.4. Geospatial Search and Route Planning

The great-circle distance between two points on a sphere can be calculated with the Haversine formula [15]. For latitudes ϕ_1, ϕ_2 and longitudes λ_1, λ_2 expressed in radians,

$$a = \sin^2\left(\frac{\phi_2 - \phi_1}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right), \tag{2.3}$$

$$d = 2R\,\mathrm{atan2}\left(\sqrt{a}, \sqrt{1-a}\right), \tag{2.4}$$

where R is an Earth-radius constant. This is a useful geographic distance, but it is not a road distance or an estimate of travel time.

The project uses this formula in two related ways. The PostgreSQL search function first applies a latitude/longitude bounding box and then evaluates Haversine distance for exact radius filtering and nearest-first ordering. The routing helper calculates direct point-to-point distance and multiplies it by a fixed factor of 1.3 to label an approximate road distance. It sets travel time to null and returns a warning; the source does not call Mapbox or OSRM for the itinerary route. This explicit uncertainty is preferable to manufacturing a drive duration, but the estimate should not be used for navigation or as proof that itinerary gaps are sufficient. The Traveling Salesperson Problem (TSP) and Orienteering Problem provide formal models for selecting and ordering destinations under travel and time limits [16], [17]. The repository does not implement an exact TSP solver or a road-network optimizer. Its fixed schedule slots establish temporal separation, and its route helper reports approximate distances after a draft is built. A future optimizer could combine road-network durations, opening-time windows, and explicit user utility, but would require reliable routing data and a defined objective.

2.5. Open Geographic Data and Data Quality

OpenStreetMap is a collaborative map database with uneven attribute coverage [1]. An open extract can cover a wide area, but it is not a curated tourism catalogue. Names, category tags, addresses, and operating details depend on local mapping practices. The repository records its Vietnam OSM shapefile as an October 2026 extract under the Open Database License. The import script selects tourism-related feature classes and filters out records without names or with coordinates outside Vietnam.

Data cleaning therefore has a direct effect on system behavior. Excluding unnamed records avoids returning blank labels, while retaining null prices and hours avoids false precision. Deduplicating source identifiers is necessary when features occur in more than one shapefile layer. At the same time, a coordinate bounding box is a coarse geographic rule: it may include offshore or border-adjacent records and does not establish administrative membership. A result whose address is only a coordinate fallback remains less useful to a traveler than a verified street address.

2.6. Comparison and Research Gap

Table 2.1 contrasts broad capability families with the evidence present in this repository. The comparison is descriptive and does not claim a controlled head-to-head experiment.

Table 2.1: Capability comparison for travel-assistant design

Approach	Strength	Limitation in the travel setting
Keyword / SQL search	Exact matching and transparent structured filters	Sensitive to wording and metadata quality; does not rank by learned preference
Dense vector retrieval	Can match semantically related phrasing [9]	Requires a configured embedding/index path and a relevance benchmark; absent from the default itinerary path
LLM-only response	Flexible natural-language interaction [3]	May state unsupported or outdated place facts; does not enforce schedule constraints by itself
Retrieval-grounded generation	Conditions text generation on retrieved evidence [4]	Quality depends on retrieval recall, source freshness, and faithful use of evidence
Constraint validator	Reproducible rejection of defined invalid states	Cannot verify rules that are not represented, such as live traffic, budget arithmetic, or all opening-hour formats
Project itinerary workflow	SQL candidate retrieval, deterministic schedule, checks, draft/confirm boundary	Does not include measured ranking quality or live road-time optimization in the inspected path

The engineering gap addressed by the repository is not that existing systems lack conversational or geographic features in general. Rather, it is the integration challenge of connecting a local geographic dataset, a web interface, a draft-generation flow, explicit validation, and authenticated persistence while preserving the distinction between known values and estimates. The main research opportunity is to evaluate whether this integration improves grounded usefulness and schedule feasibility under realistic Vietnamese travel queries. The available snapshot establishes implementable mechanisms and initial test scenarios, but it does not yet answer that empirical question.

3. Dataset and Pre-processing

3.1. Data Sources and Scope

The main geographic source is a Vietnam extract distributed as OSM shapefiles. The local README identifies it as a Geofabrik Vietnam package obtained in October 2026 and describes the files as derived from OpenStreetMap under the Open Database License. This month and year are reported as source provenance; they do not imply that the extract is current or the latest available version.

The import script, `scripts/import_all_osm_poi.py`, processes three layers:

- `gis_osm_pois_free_1`, containing named points such as restaurants, cafes, hotels, and tourism features;
- `gis_osm_pofw_free_1`, containing places of worship;
- `gis_osm_natural_free_1`, containing selected named natural features.

The source files also include administrative boundaries, roads, railways, waterways, buildings, and other layers, but the full POI importer limits ingestion to the three layers above. This reduces irrelevant records for the travel catalogue and keeps the main import process focused.

The repository also contains `data/osm_vietnam_pilot_sample.json`, a small 15-record fixture used by the regression tests. It has five observed categories, including restaurants, cafes, hotels, Buddhist temples, and mountain attractions. It is a deterministic sample for offline planning behavior, not a representative statistical sample of the national dataset.

3.2. Candidate Filtering and Standardization

The importer's allow-list maps OSM feature classes (`fclass`) into user-facing categories and broader groups. It includes dining and lodging features as well as attractions, parks, museums, places of worship, memorials, and selected natural features. The filter removes records without a display name, feature classes outside the allow-list, and coordinates outside the configured Vietnam bounding box. The implemented bounds are latitude $8.0°$ to $24.0°$ and longitude $102.0°$ to $110.0°$. These bounds are a pragmatic rectangle, not an administrative-boundary polygon.

For a retained record, the importer preserves the source layer, OSM identifier, OSM class and numeric code, name, coordinates, raw properties, and tags. It creates a standardized category and summary description. If no usable address is present, the configured fallback is the explicit Vietnamese text *Chưa có địa chỉ cụ thể; dùng tọa độ OpenStreetMap*. Unknown price, rating, opening hours, phone number, and image URL remain null. These choices preserve data provenance and avoid converting missing information into fabricated facts.

The import command defaults to an analytical dry run; the `--apply` option enables database writes. Batch size is configurable, with the documented default of 1,000. The staging table enforces a unique OSM identifier, and the importer uses an upsert strategy keyed by `osm_id` to update an existing source

record or insert a new one. The migration defines JSONB columns for tags, aliases, and raw properties, which allows source detail to be retained without flattening every OSM attribute into a separate relational column.

3.3. Recorded Dataset Profile

The committed file `data/osm_full_ingestion_summary.json` is the source for the descriptive profile in this section. It records 30,941 filtered candidates and 30,839 unique identifiers. Its layer-level figures and rejection counts are reproduced in Table 3.1. These are processing-summary values; they are not a live query of the database.

Table 3.1: Layer-level filtering values recorded in the ingestion summary

Source layer	Input	Candidates	No name	Class	Coords
OSM POIs	75,037	28,482	23,862	22,678	15
Places of worship	1,375	1,127	248	0	0
Natural features	34,009	1,332	32,596	81	0
Total	**110,421**	**30,941**	**56,706**	**22,759**	**15**

The category-group totals in the same file sum to 30,839, as shown in Table 3.2. The distribution is concentrated in food and accommodation categories. A high count of a category does not imply high quality or completeness: it is simply a count of classified identifiers in the summary.

Table 3.2: Unique POI counts by standardized category group

Category group	Records	Share
Food and dining	18,317	59.40%
Lodging	6,454	20.92%
Tourism	2,082	6.75%
Nature	1,311	4.25%
Spiritual places	1,129	3.66%
Culture	789	2.56%
History	643	2.08%
Entertainment	114	0.37%
Total	**30,839**	**100.00%**

3.3.1. A reconciliation issue in the manifest

The summary records 30,941 candidates and 30,839 unique identifiers, a difference of 102 rows. It separately reports 100 duplicate identifiers, broken down as 27 cross-layer and 73 within-layer duplicates. These fields do not reconcile arithmetically: the stated duplicate count is two lower than the difference implied by the candidate and unique totals. The category-group counts sum to the unique total of 30,839. The discrepancy is therefore reported rather than silently corrected; the importer should be rerun with a per-identifier audit before these figures are used as a final database acceptance count.

The summary also contains an overlap-analysis block with 30,839 existing staging records and 30,839 updates, while reporting zero inserts and zero deletes. That block represents the state captured by the summary process and does not independently establish the current state of a running PostgreSQL instance. The repository snapshot does not include a database export or a database query result that could verify that the staging table still contains this number of rows. The `core_tables` sub-block lists zero records for Attractions, Users, and Itineraries in the manifest's analysis; those values should not be interpreted as a present-day operational database census.

3.4. Relational Representation

The application uses SQLAlchemy models for account, chat, itinerary, and place data. Table 3.3 summarizes the primary entities visible in the ORM and the staging migration. The production-style application tables use mixed-case names such as `Users`, `Attractions`, and `ItineraryItems`, while the OSM staging table uses the lower-case name `osm_attractions_staging`.

Table 3.3: Principal relational entities in the inspected application models

Entity	Identifier and links	Purpose and notable fields
Users	Integer `user_id`; parent of conversations and itineraries	Email, password hash, language preference, verification state
Attractions	String `id`; referenced by itinerary items	Curated place details, coordinates, category, optional price and hours
Conversations	Integer `conversation_id`; foreign key to Users	Start/end time, source language, title
Messages	Integer `message_id`; foreign key to Conversations	Sender, text, translation, voice URL, JSON place list
Itineraries	Integer `id`; foreign key to Users	Selected date, title, notes, soft-delete flag
ItineraryItems	Integer `id`; links Itineraries and Attractions	Visit time, duration, notes, order index
Notifications	Integer `id`; links Users and Itineraries	Reminder text, scheduled time, sent/read-/deleted state
OTP	Integer `id`; keyed by email and purpose in service logic	Verification or password-reset code and expiry
OSM staging	Integer row ID; unique `osm_id`	OSM source/class, standardized category, GPS, JSONB attributes

The separation between `osm_attractions_staging` and `Attractions` is architecturally significant. A staging record is identified by `osm_id`; an itinerary item refers to the application's `Attractions.id`. The confirm service must therefore find or create a corresponding application attraction and must re-check the OSM record before persistence. Treating an OSM identifier as if it were already a core attraction key would break referential integrity.

3.5. Spatial and Text Pre-processing

The importer normalizes feature classes into categories, carries forward source identifiers, and applies a coordinate range check before loading records. For a radius search, `poi_search_tool.py` uses two stages. First it computes a rectangular bounding range around the search center to reduce candidate rows. It then evaluates the Haversine distance in SQL, excludes rows outside the requested radius, orders retained candidates by increasing distance, and applies the result limit after ordering. This prevents the top results from being selected by table identifier order when the request is spatial.

The distance calculation uses a mean Earth radius of 6,371 km. Given a center (ϕ_c, λ_c) and record (ϕ, λ), all angular values are converted to radians. The direct distance is

$$d = 2R \arcsin \sqrt{\sin^2\left(\frac{\phi - \phi_c}{2}\right) + \cos(\phi_c)\cos(\phi)\sin^2\left(\frac{\lambda - \lambda_c}{2}\right)}. \tag{3.1}$$

Longitude bounding width is adjusted by the cosine of the center latitude and guarded by a lower bound in the implementation. This is a search optimization; the Haversine condition remains the final radius filter.

Text search uses case-normalized `LIKE` predicates across name, category, or feature class, with SQL parameters supplied separately. It is a lexical match rather than a Vietnamese semantic search model. The chat service contains additional intent and feature extraction helpers, but the itinerary API uses its own normalized request fields. Accent normalization, query expansion, and multilingual name alias coverage should not be assumed unless present in the specific path.

3.6. Data Quality, Licensing, and Reproducibility

The data model deliberately permits unknown price, rating, hours, telephone, and image values. This improves epistemic honesty but limits itinerary personalization. If hours are null, the validator emits a warning and asks the traveler to check the schedule. It parses only a simple `HH:MM-HH:MM` form when evaluating hours; complex OSM expressions are treated as requiring manual confirmation. A missing address is represented by a fallback string rather than reverse-geocoded locality information.

OSM attribution and database licensing need to be preserved when the dataset is redistributed or publicly served. The source README identifies the ODbL and the data timestamp, but the inspected web interface and API response schema do not show a consistent user-facing attribution field for every place result. A deployment should include the required attribution and a notice describing changes to the database, while retaining a reproducible record of the extract date and importer version.

The project provides enough local artifacts to document the processing design: raw shapefiles, importer code, migration DDL, a data summary, and a compact test fixture. A repeatable dataset release should additionally record the command-line arguments, source-file checksums, Python and package versions, database row counts, and a complete duplicate reconciliation report. Those items make it possible to distinguish a change in the source extract from a change in the transformation code.

3.7. Summary

The source tree contains a concrete national OSM import path and a staging schema designed to preserve source identity and missingness. The committed processing summary describes approximately thirty thousand classified POIs, with food and lodging making up most of the classified entries. It also contains a two-record arithmetic discrepancy in its duplicate accounting. Neither the fixture nor the manifest establishes that all records are currently loaded into the configured database. These qualifications frame the system design and evaluation in the following chapters: data coverage and data quality are separate properties, and both need direct checks before a travel recommendation can be considered reliable.

4. Model Configuration

4.1. Application Architecture

The application is a single Flask service with a browser client and REST API. `main.py` loads environment variables, constructs the application with `create_app()`, starts the itinerary reminder scheduler, and runs the development server on `127.0.0.1:5000`. The application factory initializes Flask-CORS, Flask-Mail, SQLAlchemy, and Flask-RESTX, serves the `index.html` page at the root path, and registers namespaces for authentication, chat, travel search, itinerary, map, notifications, and legacy location-data operations. Interactive API documentation is exposed at `/docs`.

Figure 4.1 shows the high-level connections supported by the inspected source. It deliberately separates the language-model travel-search path from the deterministic itinerary path.

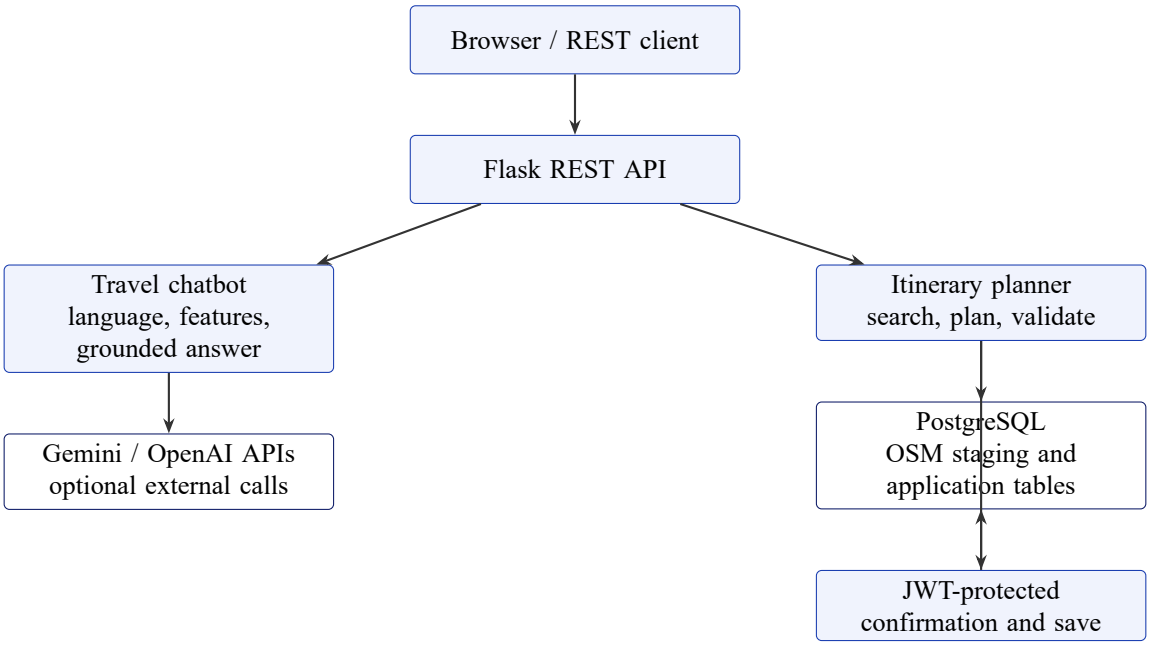


Figure 4.1: Implemented application paths and external dependencies

The browser client is implemented in HTML, CSS, and JavaScript under `src/templates` and `src/static`. It communicates with API endpoints for questions, place results, accounts, and itinerary operations. The controllers translate HTTP requests into service calls; services contain provider, search, authentication, notification, and planning logic; models map relational data to SQLAlchemy objects. This layered organization makes the core request paths identifiable, although the repository also contains legacy modules that coexist with newer itinerary components.

4.2. Natural-Language Travel Search

The `/api/travel-chatbot/search` endpoint accepts a question and an optional `enable_web_search` flag. The controller then:

1. It rejects a request without a question and calls language-detection logic.
2. The service extracts an intent and structured features from the question. The source contains provider-backed extraction and fallback behavior, as well as rule-based feature handling.
3. The search service queries PostgreSQL staging for up to six place results using the extracted feature set.
4. A response-generation function receives the original question, retrieved records, extracted features, detected language, and optional web-search setting.
5. The controller returns a structured response that can include the answer, language, POI results, suggested activities, follow-up questions, and extracted features.

The chat service is the path where current provider-based language generation is most visible. `travel_chatbot_service.py` includes a Gemini REST call, model fallback attempts, optional Google Search grounding, and a legacy OpenAI fallback. The exact path depends on keys, network access, provider responses, and the requested mode. The returned grounding metadata can include web queries and source URLs. These features should be described as conditional integrations: the repository alone does not show that credentials are set or prove that an external request succeeds in a deployed environment.

The chatbot search helper and the itinerary search tool are not interchangeable. The former uses extracted chat features and a search limit oriented to conversational responses. The latter takes explicit normalized itinerary parameters and searches `osm_attractions_staging` for planner candidates. A question answered by the chat function is not automatically stored as a travel plan.

4.3. Itinerary Draft Workflow

The itinerary API exposes both `/api/itinerary/plan-draft` and its alias `/api/itinerary/multi-agent/plan-draft`. The controller requires an ISO-formatted selected date and either a search term or coordinates. If an area label or location name is supplied without coordinates, the controller returns an input error: the source does not geocode the label in this path. When coordinates are provided, latitude and longitude must fall within a Vietnam bounding box, and the radius must be between 0.1 and 100 km. The default search radius for a coordinate-centered request is 5 km. The request also accepts a category, preferences, budget text, and a requested maximum stop count.

After input normalization, the controller calls `orchestrate_draft_itinerary()` with `execute_api=False` and at most two retries. The orchestration sequence is:

1. Validate the request date, coordinate pair, radius, result limit, and stop count.
2. Query PostgreSQL staging. If no candidates are found, return a clear not-found state rather than inventing POIs.

3. Pass the candidate records to the planner. The default planner partitions candidates into dining-related and other POIs, then assigns selected records to a fixed sequence of day-time slots.
4. Validate the draft against the original candidate list and geographic/time rules.
5. If hard errors are present, feed them back and repeat, up to the configured retry limit. Warnings alone do not trigger another attempt.
6. Return the final draft, validation report, route legs, warnings, candidate count, and iteration count. The API marks the result as unsaved.

The schedule routine uses a fixed slot template rather than learning an itinerary policy. Its current slots begin at 08:30, 10:30, 12:30, 14:45, and 18:00, with durations from 90 to 120 minutes. It attempts to alternate dining and non-dining candidates according to slot labels, avoids reusing the same `osm_id`, and limits the draft to the requested number of stops and available candidates. These are implementation details of the baseline engine; they are not evidence that the itinerary is globally optimal or personalized to a traveler.

The planner prompt builder and Gemini integration branch are present in `planner_agent.py`. The optional branch requests structured JSON and uses a low-temperature configuration. However, the endpoint supplies `execute_api=False`, so the planner uses `deterministic_planner_v1` on the default API path, regardless of whether a Gemini key happens to be configured. This distinction is central to interpreting any result from the current endpoint.

4.4. Validation and Route Estimates

The validator separates blocking errors from warnings. It verifies that each draft stop references an OSM identifier present in the candidate set and that a stop is not duplicated. It checks that coordinates are numeric and fall inside the same broad Vietnam bounds used by the planner. It checks the `HH:MM` time format, requires the end to be after the start, reconciles declared duration with the time interval, and rejects overlapping visits. The total span from the first start to the last finish cannot exceed 12 hours.

Opening-hour validation is deliberately conditional. A simple hour range can be checked against a stop interval. A missing value produces a warning; a complex value that the simple parser cannot interpret also produces a warning for manual review. The validator does not claim to fully parse the OSM opening-hours grammar.

The routing helper computes Haversine distance between consecutive stops and multiplies it by 1.3. Each result is labeled as estimated, while `travel_time_minutes` is null. The Mapbox token helper reads a token from the environment, but the route function in the inspected implementation does not make a network request to Mapbox or OSRM. When no true drive duration exists, the validator reports that the time gap is undetermined instead of passing an estimate off as a route duration. The approximate distance is useful only as a coarse indication of spatial separation.

The implemented checks do not enforce the requested budget, even though the request can carry a budget value. They also do not optimize the stop order against road travel, calculate opening hours from a weekly calendar, consider transit schedules, or validate weather and event information. Those are current scope limits, not failed tests.

4.5. Model and Provider Configuration

The project has separate model/provider settings for the conversational answer path and the itinerary planner. Table 4.1 summarizes configuration points visible in the source and environment loader. Credentials are read from environment variables; values are intentionally not reproduced in this report.

Table 4.1: Provider and database settings visible in the source configuration

Area	Configuration	Observed behavior
Chat response	`GEMINI_API_KEYS`, `GEMINI_API_KEY`, `GEMINI_MODEL`	Gemini REST integration tries a model pool; optional Google Search grounding is used when requested and supported.
Chat fallback	`OPENAI_API_KEY`	Legacy OpenAI calls are present as a fallback path in the travel chat service.
Draft planner	`GEMINI_API_KEYS`, `GEMINI_MODEL`	Gemini SDK branch exists, but the HTTP draft endpoint explicitly disables external API execution.
Map routing	`MAPBOX_ACCESS_TOKEN` or `MAPBOX_TOKEN`	Token accessor exists; itinerary routing currently returns a Haversine-based estimate.
Database	`DATABASE_POSTGRESQL_URL`	SQLAlchemy uses PostgreSQL when a database URL is configured.
Email	`MAIL_SERVER`, `MAIL_PORT`, TLS and account settings	Flask-Mail sends OTP and reminder messages when SMTP configuration is available.
Authentication	`SECRET_KEY`	Used to sign and verify application JWTs.

The optional Gemini planner uses a structured JSON response request and a temperature of 0.2. Its default model string is `gemini-3-flash-preview`, but callers can override it through configuration. If the optional SDK or provider request fails, the planner falls back to the deterministic routine. The default HTTP route avoids this dependency by never enabling the API branch.

The travel-search service uses a different provider implementation. It creates REST requests to Gemini models with a configured temperature and output-token limit, retries through a model list, and can request a Google Search tool for non-JSON generation. It may use a legacy OpenAI call if the Gemini route is unavailable. These settings do not configure itinerary drafting because the planner endpoint is deliberately offline by default.

4.6. Persistence, Accounts, and Supporting Services

Draft creation and persistence are separate operations. The draft route returns an in-memory object and marks `is_saved=False`. The confirmation endpoint is available at `/api/itinerary/confirm` and `/api/itinerary/multi-agent/confirm`. It requires a `draft_id`, selected date, and non-empty items list, then derives the account identity from the Bearer JWT rather than trusting a user identifier in the request body. It verifies the token signature and expiry, looks up the user, and rejects unverified accounts. The confirmation service validates the request against staging data and commits the application itinerary through the relational model.

The account endpoints include registration, email verification with a one-time code, login, password-reset steps, and username update. The user model stores a password hash rather than a plaintext password. The chat models store conversations and messages; itinerary entities link the user, trip date, and ordered attraction visits. Notifications can be queried, marked read, and deleted through their own namespace.

The application factory calls `db.create_all()` inside an app context, but catches connection errors and logs a warning. The repository also provides SQL migrations for the OSM staging and notifications tables. This combination supports local development, but it does not constitute a versioned migration framework for every model change; deployment procedures should apply and verify migrations explicitly.

The scheduler in `scheduler_service.py` is started when `main.py` runs. It periodically checks upcoming itineraries and calls the email service. The development entry point enables Flask debug mode and binds to localhost. That is appropriate for local use but should be changed for production deployment. The Docker Compose file provides a PostgreSQL-compatible development container using the pgvector image, but the itinerary query path described here uses relational columns and does not require vector-extension operations.

4.7. Security and Operational Considerations

The project reads database, mail, model, map, and signing credentials from environment configuration. The report does not expose local `.env` values. A deployment should keep the environment file outside source control, rotate any credentials that have been exposed, use a strong signing key, and limit access to the database. HTTP provider integrations introduce cost, rate-limit, availability, and data-transfer considerations. User questions can contain personal context, so provider calls should be disclosed and minimized according to the deployment's privacy requirements.

The confirmation flow has a useful identity boundary: the server derives the user from the signed token and does not accept the request body's `user_id` as authority. Still, token, OTP, registration, and password-reset behavior should be independently reviewed for expiry, rate limiting, brute-force protection, and secure transport before public deployment. Database connection success, migration status, provider availability, and SMTP delivery should be observable with structured logs that exclude secrets and unnecessary personal content.

SQLAlchemy parameters are used for the itinerary search values, and the staging migration declares a unique constraint on `osm_id` plus indexes for coordinates, category, feature class, source layer, and import status. These support expected retrieval and idempotent ingestion patterns. Performance at scale remains unmeasured in the repository snapshot and must be tested against a provisioned database with recorded hardware and configuration.

4.8. Development Configuration

The local setup described by the repository creates a Python virtual environment, installs `requirements.txt`, starts the development database with `docker compose -f docker-compose.dev.yml up -d`, configures required environment variables, applies the relevant SQL migration, imports the POIs with an explicit `--apply` flag, and starts the server with `python main.py`. The developer interface is available at the root path and Swagger documentation at `/docs`.

Because the importer defaults to dry run and the draft endpoint defaults to deterministic offline execution, the core dataset-processing and itinerary-drafting path can be developed without model-provider API calls. Database-backed behavior still requires a reachable PostgreSQL instance with the staging table available. Optional chatbot generation, SMTP, and map services require their respective credentials and network access.

5. Experimental Results and Discussion

5.1. Evaluation Scope and Evidence Available

A quantitative evaluation should be grounded in a fixed dataset, labeled queries, a defined system configuration, and recorded execution output. The repository snapshot contains a data-processing summary, a small pilot fixture, implementation code, and a test module. It does not contain a labeled POI retrieval benchmark, a response-quality annotation set, output logs from an LLM evaluation, or a reproducible load-test report. This chapter therefore reports the evidence that is present and describes a protocol for collecting the evidence that is missing.

The distinction matters because three types of statement are often confused. First, a source-code rule (for example, rejecting a repeated `osm_id`) is an implemented behavior. Second, a unit-test assertion is an intended check, and only an executed test run can establish whether the check passed for that revision. Third, a model metric such as answer accuracy or route feasibility requires a defined test corpus and measured outputs. The current repository supports the first two at source level; it does not document the third.

5.2. Dataset Processing Results

The committed processing summary records 110,421 input records across the three imported layers. Of these, 30,941 rows pass layer-level candidate filtering and 30,839 unique identifiers are listed after deduplication. The summary provides category counts and layer-level rejection counts, reproduced in Chapter 3. Those values are useful as a descriptive data profile and show that the import pipeline performs substantial name/class/coordinate filtering.

The manifest's counts are not fully self-consistent. Subtracting 30,839 unique IDs from 30,941 candidates gives 102 duplicate rows, whereas the explicit duplicate field reports 100. The grouped category counts do sum to 30,839. This inconsistency is a data-quality finding, not a corrected result. A fresh import audit should emit each duplicate OSM ID and the source layers in which it occurs, then reconcile the summary before presenting a final accepted-row count.

The summary lists null values for unknown price, rating, and opening hours, and defines a coordinate-based fallback address. This is a positive design choice because the data pipeline does not substitute made-up values. The trade-off is that the catalogue's usefulness varies by field: a point can be geographically searchable while lacking the details needed to answer whether it is open, affordable, or appropriate for a visit. The repository does not publish field-level completeness percentages, so completeness rates cannot be calculated from the evidence reviewed.

5.3. Functional Test Design in the Repository

The file `tests/test_multi_agent_flow.py` defines nine unittest methods. Its module docstring states that the tests use the pilot fixture and mocked database/provider calls and do not connect to or

Table 5.1: Regression test cases declared in the repository

No.	Test scenario	Expected behavior encoded by the assertion
1	Offline planner with flat pilot coordinates	Successful validated draft, three stops, and numeric coordinates
2	Plan-draft endpoint with mocked POI results	HTTP 200, valid draft, and `is_saved=False`
3	Invalid selected-date text	HTTP 400
4	Date in the past	HTTP 400 and invalid-request status
5	Latitude without longitude	HTTP 400
6	Non-numeric coordinate input	HTTP 400
7	Simulated staging database error	HTTP 503 and database-error status
8	Confirm endpoint without Bearer token	HTTP 401
9	Login with mocked auth service	HTTP 200 and the mocked token/user payload

write to the configured PostgreSQL database. Table 5.1 summarizes the assertions represented by those methods.

These cases target input validation, the offline planning path, API status mapping, and selected authentication behavior. The test module does not exercise a live PostgreSQL connection, actual transaction rollback, email delivery, an external language-model provider, Google Search grounding, actual road routing, or concurrent load. It does not score response relevance or truthfulness. This is appropriate for a small safe regression suite, but it should not be cited as evidence of production reliability or model quality.

The tests were inspected as source and were not executed while preparing this report. The assertions in Table 5.1 therefore describe expected behavior encoded in the test file; this report does not assign a pass/fail outcome to the current checkout.

5.4. Implemented Behavior Observed by Source Inspection

Several properties can be established from the implementation without claiming benchmark results:

- The itinerary endpoint sets `execute_api=False`; its normal draft path therefore uses the deterministic scheduling routine.
- The orchestrator passes only database-retrieved or explicitly supplied candidate POIs to the planner, and the validator checks draft identifiers against that candidate set.
- The route helper returns a Haversine-based distance estimate multiplied by 1.3 and leaves travel time unknown.
- The validator separates errors that invalidate a draft from warnings about missing or complex opening-hour information and unknown travel time.
- The draft endpoint returns an unsaved object. Confirmation is a separate operation guarded by a verified-user JWT lookup.

- The radius search performs a bounding-box prefilter, exact Haversine radius check, distance ordering, and then limiting.

These properties explain the software's safety boundaries, but they do not quantify their real-world effectiveness. For example, checking that a drafted `osm_id` belongs to the candidate set prevents a planner from adding an identifier outside that set. It does not verify that an OSM name is accurate, that a POI is still operating, or that the candidate set itself contains the best locations.

5.5. Proposed Retrieval Evaluation

A future retrieval study should begin with a manually labeled query set. Each query should have one or more relevant OSM IDs and a reason for relevance, with categories covering exact place names, categories, text-only area labels, coordinate-centered searches, Vietnamese diacritics, and multilingual wording. Each query should be run against a frozen database snapshot and a recorded limit and radius. The relevance label must be prepared independently of the search results to avoid scoring the system against its own output.

For a query set $\mathcal{Q}$, the following ranking measures are suitable:

$$\text{Hit@K} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbb{I}\left(R_q^{(K)} \cap G_q \neq \varnothing\right), \tag{5.1}$$

$$\text{MRR} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{\text{rank}_q}, \tag{5.2}$$

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}, \tag{5.3}$$

where $R_q^{(K)}$ is the retrieved top-K, G_q is the set of relevant IDs, and rank_q is the first relevant position. Hit@K measures whether at least one known relevant item appears; MRR rewards earlier first relevant results; NDCG accommodates graded relevance when annotators distinguish exact, close, and weak matches. None of these scores is available in the current snapshot.

At minimum, a retrieval evaluation should compare the current SQL text/category search against a geographic-only baseline and a clearly specified semantic-search baseline if the optional Chroma index is intended for deployment. All approaches must use the same frozen catalogue and query set. Results should be reported by query type and language, since an aggregate score can conceal poor performance on local names, administrative labels, or non-Vietnamese queries. A ranking score alone should not be interpreted as factual response accuracy.

5.6. Proposed Itinerary and Answer Evaluation

For itinerary drafts, a useful primary measure is the independently audited feasibility rate:

$$\text{FeasibilityRate} = \frac{\text{drafts meeting all predefined hard constraints}}{\text{drafts reviewed}}. \tag{5.4}$$

The hard-constraint checklist should include candidate-set membership, duplicate stops, coordinate validity, valid time intervals, interval overlap, daily span, and any enforceable opening-hour rules. Because

the validator implements only a subset of real travel constraints, a human audit or independent checker should also consider route plausibility, accessibility, opening calendars, and available time. A validator's own `valid=True` output cannot serve as the sole ground truth for measuring that validator.

Grounded answer quality should be assessed at the claim level. Reviewers can label each factual claim as supported by the retrieved OSM record, supported by a cited web source, unsupported, or contradicted. This makes it possible to distinguish a correct place name from a fabricated price or an unqualified opening-hour statement. Human ratings of relevance, clarity, and language quality can supplement automated text similarity. BLEU and ROUGE are available as established text-overlap measures [18], [19], but are insufficient by themselves for open-ended travel advice: a faithful paraphrase may have low overlap, while a fluent answer may repeat reference words and still contain a false claim.

Route evaluation requires a reference routing source and a declared travel mode. For each stop pair, the system's estimated distance can be compared with a road-network distance and the error distribution reported. Since the current route helper returns no estimated duration, travel-time error cannot yet be calculated for the default itinerary path. If a road-routing API is integrated, evaluation should record its date, mode, avoidances, and API version because these affect routes and traffic assumptions.

5.7. Proposed Runtime and Reliability Evaluation

No latency, throughput, provider error, or load-test measurements are included in the repository snapshot. A reproducible runtime study should name the host CPU/RAM, Python version, dependency lock, PostgreSQL version and row count, network conditions, model/provider and region, concurrency schedule, warm-up duration, and total request count. It should separate database-only search, deterministic draft generation, live provider-backed generation, and confirmation transactions. Those routes have different cost and latency profiles and should not be aggregated into one number.

For each endpoint and concurrency level, report median and 95th-percentile latency, completed requests per second, status-code counts, timeout rate, database pool saturation if configured, and external-provider failures. Repeat each run and report variability. A load test against a mocked model does not establish the behavior of an external provider, and a local developer server with Flask debug enabled is not representative of a production WSGI deployment.

5.8. Discussion of Current Results

The available evidence supports a modest conclusion. The data-preparation summary describes a broad filtered candidate set; the SQL model and query tool provide a concrete geographic search mechanism; the draft planner has a deterministic offline execution path; and the validator encodes several useful structural and temporal safeguards. A safe, mocked regression suite is present for representative request and response cases. This is meaningful implementation evidence for a capstone prototype.

The evidence does not support the numerical claims found in the earlier draft of this report about retrieval hit rate, multilingual BLEU/ROUGE scores, itinerary feasibility percentages, zero hallucination, or concurrent throughput. The repository contains no corresponding labeled test set or run log. Such values have been removed from the report rather than reused as if they were measured. Similarly, the presence of `chromadb`, `sentence-transformers`, and `pgvector` in dependencies or the container image does not establish a working vector retrieval benchmark in the default itinerary flow.

Several implementation constraints directly affect interpretation:

1. **Planner personalization is limited.** The baseline uses a fixed slot schedule and coarse dining classification. User preference and budget fields are accepted but not comprehensively scored or validated.
2. **Retries may not improve the draft.** The orchestrator can return validator feedback for up to two retries. The deterministic selection routine does not perform an explicit search over alternatives in response to each error, so retries are a control-flow feature whose practical value should be tested.
3. **Road feasibility remains unknown.** Approximate distances are reported without drive times. Time gaps between visits cannot be certified as sufficient for travel in the default path.
4. **Data freshness and detail vary.** Null hours and prices limit concrete guidance; a count of records does not establish that records are current or tourism-relevant.
5. **The manifest has an unresolved count mismatch.** A release-level import audit is needed to reconcile duplicate rows and verify the deployed staging-table count.

5.9. Recommended Evaluation Sequence

The next evaluation cycle should proceed in an order that limits confounding:

1. Rerun the importer in dry-run mode from the checked-in OSM extract; produce per-layer counts, an explicit duplicate-ID file, and a reconciled final count.
2. Load a frozen copy into a test PostgreSQL database, record the schema version, and query total rows and field completeness.
3. Execute the existing unittest module and retain the terminal log and revision identifier. Then add direct tests for validator boundary conditions and confirm-service transaction behavior.
4. Create a blinded, manually labeled query set and score text, category, and radius retrieval separately.
5. Evaluate deterministic itinerary drafts on a fixed request set. Compare validator output with an independent human or reference-rule audit, and report each constraint separately.
6. Evaluate the provider-backed chat and planner only after recording model names, prompts, parameters, provider region, and exact request/response outputs under a privacy-safe protocol.
7. Conduct performance and failure testing against a production-like WSGI process and database, reporting the configuration alongside all latency and error results.

5.10. Chapter Summary

The current snapshot offers descriptive data-processing figures, concrete source-level behavior, and nine declared regression tests. It does not offer executed test results or measured model and system benchmarks. The strongest defensible result is therefore that a database-grounded, offline draft-and-validation workflow is implemented with explicit uncertainty around missing opening hours and road travel time. The main empirical work still needed is to verify the imported dataset, measure retrieval relevance, independently audit itinerary feasibility, and benchmark each runtime path separately.

References

[1] M. Haklay and P. Weber, "Openstreetmap: User-generated street maps," *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12–18, 2008.

[2] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.

[3] T. B. Brown, B. Mann, N. Ryder, *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.

[4] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[5] Y. Gao, Y. Xiong, X. Gao, *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[6] J. Borràs, A. Moreno, and A. Valls, "Intelligent tourism recommender systems: A survey," *Expert Systems with Applications*, vol. 41, no. 16, pp. 7370–7389, 2014.

[7] D. Gavalas, C. Konstantopoulos, K. Mastakas, and G. Pantziou, "Mobile recommender systems in tourism," *Journal of Network and Computer Applications*, vol. 39, pp. 319–333, 2014.

[8] K. H. Lim, J. Chan, C. Leckie, and S. Karunasekera, "Personalized tour recommendation based on user interests and points of interest popularity," *Artificial Intelligence Review*, vol. 52, no. 4, pp. 2725–2749, 2019.

[9] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982–3992, 2019.

[10] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[11] Q. Wu, G. Bansal, J. Zhang, *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[12] S. Hong, M. Zhuge, J. Chen, *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," *arXiv preprint arXiv:2308.00352*, 2023.

[13] S. Yao, J. Zhao, D. Yu, *et al.*, "ReAct: Synergizing reasoning and acting in language models," *International Conference on Learning Representations (ICLR)*, 2023.

[14] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 8634–8652, 2023.

[15] R. W. Sinnott, "Virtues of the haversine," *Sky and Telescope*, vol. 68, no. 2, p. 158, 1984.

[16] D. L. Applegate, R. E. Bixby, V. Chvatal, and W. J. Cook, *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.

[17] P. Vansteenwegen, W. Souffriau, and D. V. Oudheusden, “The orienteering problem: A survey,” *European Journal of Operational Research*, vol. 209, no. 1, pp. 1–10, 2011.

[18] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: A method for automatic evaluation of machine translation,” *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 311–318, 2002.

[19] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, 2004, pp. 74–81.

A. Data Dictionary and Reproducibility Notes

A.1. Staging Record Fields

The OSM staging model preserves a source-facing record rather than pretending every OSM feature has the detail of a curated attraction. Table A.1 describes the key columns defined by `OsmAttractionStaging` and the staging migration.

Table A.1: OSM staging field dictionary

Field	Type / constraint	Meaning
`id`	Integer primary key	Internal staging row identifier.
`osm_id`	String, unique, not null	Stable source feature identifier used for deduplication and traceability.
`osm_source_layer`	String, not null	Name of the source shapefile layer.
`osm_code`	Integer, optional	Numeric source class code when available.
`fclass`	String, not null	Original OSM feature class.
`name`	String, not null	Display name retained from the source record.
`category`	String, not null	Normalized travel-facing category.
`latitude`, `longitude`	Double precision, not null	Geographic coordinates in WGS 84 decimal degrees.
`address`	String, not null	Source address when available; otherwise explicit coordinate-based fallback text.
`description`	Text, optional	Brief description derived from retained OSM information.
`price`, `rating`	Numeric, optional	Unknown values remain null in the import standard.
`opening_hours`	String, optional	OSM hours string; format may be complex or absent.
`phone_number`, `image_url`	String, optional	Contact and image metadata when present.
`language`	String	Language label assigned by the import process.
`tags`, `aliases`	JSONB	Searchable tags and alternative names.
`raw_properties`	JSONB	Original property values retained for traceability.
`imported_to_attraction`, `attraction_id`	Boolean and string	Promotion state and optional link to the core application attraction.

Field	Type / constraint	Meaning
`created_at`, `updated_at`	Timestamp	Row creation and update times.

A.2. Import Reproduction Checklist

The repository's import script is designed to support a dry-run review before writes are enabled. A release record should include:

1. the extract timestamp and license notice from the source package;
2. file checksums for the three input layers;
3. the exact Python version and installed dependency versions;
4. importer options, including deduplication mode and batch size;
5. raw row counts, per-filter rejection counts, and retained counts by layer;
6. a machine-readable duplicate report listing each repeated OSM identifier;
7. a final staging row count and a sample of records queried from PostgreSQL;
8. the database schema or migration revision used for the load.

This checklist would resolve the current ambiguity between the summary's candidate, unique, and duplicate counts. A safe release should also retain the dry-run report alongside the applied-run report so that proposed changes can be compared with actual writes.

B. API Surface and Request Boundaries

B.1. Core Travel and Itinerary Routes

Table B.1 summarizes the routes central to the project topic. The route paths and operation purpose are derived from the registered Flask-RESTX namespaces and controllers. The interactive Swagger page at `/docs` is the primary runtime reference for detailed schemas and status responses.

Table B.1: Core travel-search and itinerary endpoints

Method	Path	Purpose / boundary
POST	`/api/travel-chatbot/search`	Accepts a natural-language travel question and an optional web-search flag; returns search results and a generated response.
GET	`/api/travel-chatbot/metadata`	Reads paginated place metadata through the travel-chatbot controller.
GET	`/api/travel-chatbot/embeddings`	Legacy embeddings route retained in the controller; it is not the default itinerary retrieval mechanism.
POST	`/api/itinerary/plan-draft`	Generates a draft. Requires a selected date and either a text query or a valid coordinate pair; does not persist the draft.
POST	`/api/itinerary/multi-agent/plan-draft`	Alias for the draft-planning operation.
POST	`/api/itinerary/confirm`	Confirms and persists a draft using the verified account from a Bearer JWT.
POST	`/api/itinerary/multi-agent/confirm`	Alias for the confirmation operation.
GET / POST / DELETE	`/api/itinerary/...`	Existing list, detail, create, delete, and item-update operations for persisted itineraries. Consult Swagger for the exact method and payload of each resource.

B.2. Draft Request Validation

The plan-draft controller accepts a date, optional free-text query, destination or area label, category, coordinates, radius, preferences, budget, maximum stop count, and an optional user prompt. It rejects a partial coordinate pair, non-numeric or out-of-range coordinates, a radius outside the supported interval,

and an area label without a coordinate center. This last rule is important: the itinerary controller does not translate an area name into a coordinate.

The orchestrator further validates request-level limits, including a stop count from one through ten and a search result limit from one through one hundred. A date in the past is rejected in the orchestration layer. Search failures are mapped to explicit response statuses; database unavailability is reported separately from invalid input and a valid empty search.

B.3. Draft Confirmation Boundary

The confirmation route requires a draft identifier in the form expected by its controller, a selected date, and a non-empty list of itinerary items. It reads the Bearer token from the authorization header, verifies its signature and expiry, obtains the authenticated user, and checks that the user's email is verified. A user identifier included in the JSON body is not accepted as the authority for ownership.

The confirmation service rechecks draft details against staging records and persists a relational itinerary. A successful new confirmation returns HTTP 201; a recognized duplicate request can return HTTP 200. Validation errors, missing data, authorization failures, and transaction errors are represented by distinct response categories. These controller-level response codes are part of the API contract and should remain stable if the planning implementation changes.

C. Regression Test and Evaluation Register

C.1. Existing Regression Cases

The nine test methods in `tests/test_multi_agent_flow.py` form a compact offline regression layer. The fixture contains 15 OSM-shaped records and allows planner behavior to be exercised without a production database. Provider and database functions are patched where the test needs to isolate controller behavior. The cases should be run in a clean test environment with test-specific configuration and without relying on a user's live database.

Table C.1: Test-to-requirement mapping for the current test module

ID	Requirement area	Assertion represented in the source
T-01	Offline planning	The planner returns a successful valid draft with three stops and coordinates from the flat pilot fixture.
T-02	Draft API contract	A mocked staging search produces HTTP 200 and a response marked unsaved.
T-03	Input validation	An invalid date string is rejected with HTTP 400.
T-04	Input validation	A past selected date is rejected with HTTP 400.
T-05	Coordinate validation	Supplying latitude without longitude is rejected with HTTP 400.
T-06	Coordinate validation	Non-numeric coordinate values are rejected with HTTP 400.
T-07	Failure mapping	A mocked staging database error maps to HTTP 503.
T-08	Confirmation authorization	A confirmation request without a Bearer token maps to HTTP 401.
T-09	Login controller	A mocked authentication service response is returned as HTTP 200 with token and user fields.

The test names communicate expected behavior, but a test's presence does not show that it passes on every revision. A test report should record the checkout revision, Python version, command, start time, total tests, failures/errors, and captured output. The report should not include tokens, database passwords, personal user records, or provider payloads that contain sensitive data.

C.2. Additional Tests Recommended Before Release

The current tests focus on happy-path draft creation and several request boundaries. A broader regression suite should cover these areas:

- **Search correctness:** compare SQL radius results with an independent Haversine calculation; verify nearest-first order and ensure the limit is applied after radius filtering.
- **Data ingestion:** test duplicate IDs across and within layers, malformed coordinates, empty names, category mappings, null preservation, and upsert idempotency against a disposable database.
- **Validator boundaries:** test midnight edge cases, malformed time values, zero-length visits, overlap at equality, the 12-hour maximum, simple opening-hour ranges, complex hours, and missing coordinates.
- **Planner semantics:** verify uniqueness, stop-count limits, candidate coverage, deterministic output for a fixed fixture, and behavior when dining or sightseeing categories are absent.
- **Retries:** construct a candidate set that triggers a hard error and verify whether feedback changes the second attempt or merely repeats the original plan.
- **Persistence:** verify that confirmation revalidates current staging data, maps OSM IDs to core attractions, assigns the authenticated user, and rolls back all writes on a forced failure.
- **Provider integration:** use provider mocks to test malformed structured output, quota responses, timeout, fallback, missing credentials, and safety of logs.
- **Access control:** test expired, malformed, and tampered JWTs; unverified users; attempts to confirm another user's draft; and OTP expiry and rate limits.

C.3. Evaluation Record Template

When measurements are collected, the results table should include enough context to be independently interpreted. Table C.2 is a blank record template, not a report of completed measurements.

C.4. Local Development Commands

The project README describes the following broad development sequence. Commands should be run from the repository root, after local environment values have been configured:

```
python -m venv venv
pip install -r requirements.txt
docker compose -f docker-compose.dev.yml up -d
python scripts/import_all_osm_poi.py --apply --batch-size 1000
python main.py
```

Listing C.1: Local development sequence documented by the project

The importer should first be run without `--apply` when reviewing a new source extract. The current repository's migrations should be checked and applied before the import. The development server should not be exposed publicly with debug mode enabled. For reproducible experiments, capture command output and configuration metadata separately from secrets.

Table C.2: Evaluation record template for a future run

Field	Value to record
Source revision	Git commit or archive identifier
Dataset version	OSM extract timestamp, file checksums, final database row count
Environment	OS, CPU, memory, Python and PostgreSQL versions
Provider settings	Model ID, region, temperature, output limit, grounding mode, prompt revision
Query set	Number of queries, languages, category mix, annotation procedure
Search metrics	Hit@K, MRR, NDCG by language, category, and location-query type
Itinerary metrics	Independent feasibility rate and per-constraint violations
Answer review	Supported factual claims, unsupported claims, relevance and clarity ratings
Runtime metrics	p50 / p95 latency, throughput, error and timeout rates by endpoint
Repetitions	Number of independent runs and variance or confidence interval method
Limitations	Missing labels, provider outages, data gaps, and deviations from protocol

C.5. Closing Note

This register is intended to make the report's evidence boundary easy to audit. Current source artifacts describe a test design and a dataset summary. Future execution logs and labeled evaluations should be added as artifacts with versions and methodology, then used to replace planned metrics with measured results. Until those records exist, numerical model-quality or throughput claims should remain out of the final report.

6. Instructor Review and Comment

Table 6.1: Capstone Project Evaluation Rubric and Instructor Feedback

CATEGORY	SCORE	REVIEW AND COMMENT
IDEA	____/10	
APPLICATION	____/30	
RESULT	____/30	
PROJECT MANAGEMENT	____/10	
PRESENTATION AND REPORT	____/20	
TOTAL	____/100	

Instructor Signature Date

_____ _____

TS. Trần Quang Quý